UNDER REVIEW

RESCIENCE C

Replication / Machine Learning

[Re] Communication-Efficient Language Model Training Scales Reliably and Robustly: Scaling Laws for DiLoCo

Andrew Gyakobo[1] **and Felix Schmidt**[2]

[1] Independent Researcher, New Jersey, USA – [2] Independent Researcher, Florida, USA

Edited by
(Editor)

Received
–

Published
–

DOI
–

1 Introduction

DiLoCo [1] trains a language model with k workers that synchronize only once every H optimization steps. Between synchronizations each worker trains its own copy with an *inner* optimizer (AdamW); at a synchronization the workers' average change in parameters, the *pseudo-gradient*, is applied to a shared model by an *outer* optimizer (SGD with Nesterov momentum). Communication therefore drops by a factor of H relative to synchronous data-parallel training.
Charles et al.[2] studied how DiLoCo scales with model size, from 35M to 10B parameters, and along the way reported how its hyperparameters behave. We replicate five of those findings, all qualitative and all stated for small or every model size:

- **C1** Evaluation loss increases with the sync interval H.
- **C2** The optimal outer learning rate increases with H (which the authors call "potentially counter-intuitive").
- **C3** The optimal outer learning rate increases with the number of workers M (our k).
- **C4** At small model sizes, DiLoCo falls further behind data-parallel training as the number of workers grows (at 35M parameters and $H = 30$: 0.7%, 2.0% and 3.9% worse for 2, 4 and 8 workers).
- **C5** DiLoCo with a single worker attains lower evaluation loss than data-parallel training.

We did not attempt to replicate the scaling laws themselves, which require models up to 10B parameters, nor their batch-size and wall-clock results.
Our setting is deliberately much smaller than the original: a 30M-parameter model trained from scratch on one RTX 3070 Ti (8 GB), with workers simulated one after another. The question is whether these findings survive in a setting that a student or independent researcher can run, which is also where new low-communication methods are often prototyped. Four findings reproduce, one of them only at short sync intervals. The fifth (C2) reproduces at a fixed token budget, but targeted experiments show that the variable behind it is the number of outer rounds, which shrinks as H grows, rather than H itself.

Copyright © 2026 A. Gyakobo and F. Schmidt, released under a Creative Commons Attribution 4.0 International license.
Correspondence should be addressed to Andrew Gyakobo (andygyakobo@gmail.com)
The authors have declared that no competing interests exist.
Code is available at https://github.com/Gyakobo/building-gpt/tree/main/diloco..

2 Methods

2.1 Implementation

We wrote DiLoCo from scratch in PyTorch on top of a GPT-2-style model in the style of nanoGPT [3]; the original study's code is not public. In each outer round, every worker copies the global parameters, takes H AdamW steps on its own data shard, and the outer step applies the averaged pseudo-gradient $\Delta = \theta - \frac{1}{k}\sum_i \theta_i$ with Nesterov momentum 0.9. Each worker keeps its own AdamW state across rounds. Because every worker restarts from the global parameters each round, the k workers can run sequentially on one GPU with one physical model and k optimizer states; the result is mathematically identical to running them in parallel, so we measure optimization quality per round, not wall-clock speed.

A test suite (10 tests) checks the implementation. The central test uses the fact that with $k = 1$, outer learning rate 1 and no momentum, the outer step sets $\theta \leftarrow \theta_1$, so DiLoCo must equal plain AdamW for any H: validation losses agree to within 10^{-4} for $H \in \{1, 5\}$ (largest observed difference 4.8×10^{-7} on CPU), while an outer learning rate of 0.9 changes them by about 3×10^{-2}. Other tests check that an outer learning rate of 0 freezes the model, that workers see different data, and that every run is scored on the same validation sequences.

2.2 Setup

Model and data. 6 layers, 6 heads, width 384, context 256, tied embeddings, no dropout: 30.0M parameters, of which about 10.7M are outside the embeddings. Data is TinyStories [4] tokenized with the GPT-2 tokenizer (474M training tokens); each worker samples 256-token windows from its own contiguous slice.

Optimization. 32 sequences (8,192 tokens) per worker per step; AdamW with $\beta = (0.9, 0.95)$, peak learning rate 6×10^{-4}, 100 warm-up steps (10% of the run for runs under 1,000 steps), cosine decay to 6×10^{-5}, weight decay 0.1, clipping 1.0; bfloat16 autocast.

Budgets. The main budget is 4,000 inner steps per worker, about 4 tokens per parameter for $k = 4$. To separate H from the number of outer rounds T, we also train every $H \in \{25, 100, 250, 500\}$ for exactly $T = 24$ rounds ($24H$ steps), and $H = 100$ for $T \in \{4, 8, 16, 24, 40\}$. Long runs use 18,000 steps, about the 20 tokens per parameter of the original study (each token is seen about 1.25 times).

Baselines. Each DiLoCo run is compared with plain AdamW for the same number of steps: a *sync* baseline with batch $32k$, which sees the same tokens and stands in for data-parallel training, and a *solo* baseline with batch 32, one worker alone, which also sees the same tokens as a single DiLoCo worker. We report the share of the solo-to-sync gap that DiLoCo closes: $(L_{\text{solo}} - L)/(L_{\text{solo}} - L_{\text{sync}})$.

Evaluation and seeds. Validation loss (nats per token) on a fixed set of 20×32 sequences from the TinyStories validation split. Headline configurations and comparisons that decide a claim use 2–3 seeds; seed-to-seed standard deviations are 0.0001–0.0045.

2.3 Differences from the original study

- **Scale and data.** Their smallest model has 35M parameters and is trained on web text at about 20 tokens per parameter; ours has 30M parameters (with a much smaller non-embedding part) and is trained on TinyStories, mostly at 4 tokens per parameter.
- **Hyperparameter search.** They re-tune the outer learning rate per configuration on $\{0.2, 0.4, \ldots, 1.0\}$; we sweep $\{0.1, 0.3, 0.5, 0.7, 1.0\}$ (plus 1.3 for the worker sweep). We did not re-tune the inner learning rate, including for the larger batches of the sync baselines.

- **Sync intervals.** They use $H \in \{1, 5, 10, 30, 100, 300\}$; we use $\{25, 50, 100, 250, 500\}$.
- **Workers.** Simulated sequentially, so no communication cost is modelled.

3 Results

Table 1 summarizes the outcome. Table 2 gives the headline configurations.

Table 1. Claims of the original study and the outcome of this replication.

Claim	Outcome
C1 Loss grows with H	**Reproduced** at every outer learning rate (Fig. 1).
C2 Best outer lr grows with H	**Reproduced at a fixed budget, but explained by the number of rounds**: with 24 rounds 0.7 is best at every H, and at fixed $H = 100$ fewer rounds favor 1.0 (Figs. 2, 3).
C3 Best outer lr grows with workers	**Reproduced**: the loss curve shifts toward larger values from 2 to 8 workers (Fig. 2c; one seed).
C4 Gap to data-parallel widens with workers	**Reproduced**: 2.5%, 5.4%, 8.1% above the sync baseline for 2, 4, 8 workers at $H = 25$ (Table 3).
C5 One worker beats data-parallel	**Partly reproduced**: 0.6% better than AdamW at $H = 25$, 1.8% worse at $H = 100$ and 500 (Table 4).

Table 2. Headline results after 4,000 steps with $k = 4$ (mean ± standard deviation over 3 seeds).

Method	H	Outer lr	k	Comm. rounds	Val loss	Gap closed
Sync baseline	1	–	4	4,000	1.697 ± 0.0008	100%
Solo baseline	–	–	1	0	1.983 ± 0.0009	0%
DiLoCo	25	0.7	4	160	1.789 ± 0.0045	68%
DiLoCo	100	0.7	4	40	1.887 ± 0.0041	33%
DiLoCo (original H)	500	0.7	4	8	1.936 ± 0.0019	16%

3.1 C1: loss grows with the sync interval

At 4,000 steps with 4 workers, loss increases monotonically with H at every outer learning rate (Fig. 1); at 0.7 it rises from 1.790 at $H = 25$ to 1.935 at $H = 500$. With three seeds, $H = 25$, 100 and 500 close 68%, 33% and 16% of the gap between the solo and sync baselines (Table 2). The finding also holds with the number of rounds fixed at 24: the share of the gap closed falls from 46% at $H = 25$ to 32%, 18% and -4% at $H = 100$, 250 and 500 (Table 5).

3.2 C2: the best outer learning rate follows the number of rounds

At a fixed budget of 4,000 steps, the best outer learning rate drifts upward with H, as reported: over three seeds, 1.0 beats 0.7 by 0.006 at $H = 500$, the two tie at $H = 250$, and 0.7 is better by 0.005–0.007 at $H \leq 100$ (Fig. 2a). At a fixed budget, however, larger H also means fewer outer rounds (160 at $H = 25$, only 8 at $H = 500$), and the original H sweep appears to share this coupling.

We separated the two variables in both directions. With every H trained for exactly 24 rounds, the drift disappears and 0.7 is best at every H from 25 to 500 (Fig. 2b, Table 5); at $H = 500$, 0.7 now beats 1.0 by 0.014 (two seeds each). With $H = 100$ fixed and the number of rounds varied, the preference moves: 1.0 beats 0.7 by 0.101, 0.010 and 0.003

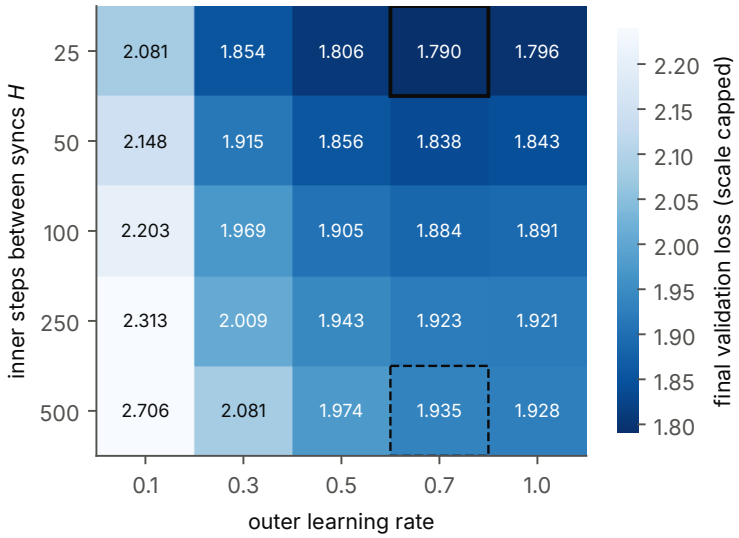


Figure 1. Final validation loss by sync interval H and outer learning rate ($k = 4$, 4,000 steps, seed 0). The solid box marks the best cell and the dashed box the setting of Douillard et al.[1].

with 4, 8 and 16 rounds and loses by 0.003 and 0.007 with 24 and 40 rounds. Across every configuration in which both values were run (Fig. 3), 1.0 wins only with 16 or fewer rounds and 0.7 wins in every run with 24 or more, whatever H and the number of steps. We therefore read C2 as a consequence of the number of outer rounds. The crossover point is specific to our setting; the runs with few rounds are short and most points use one seed.

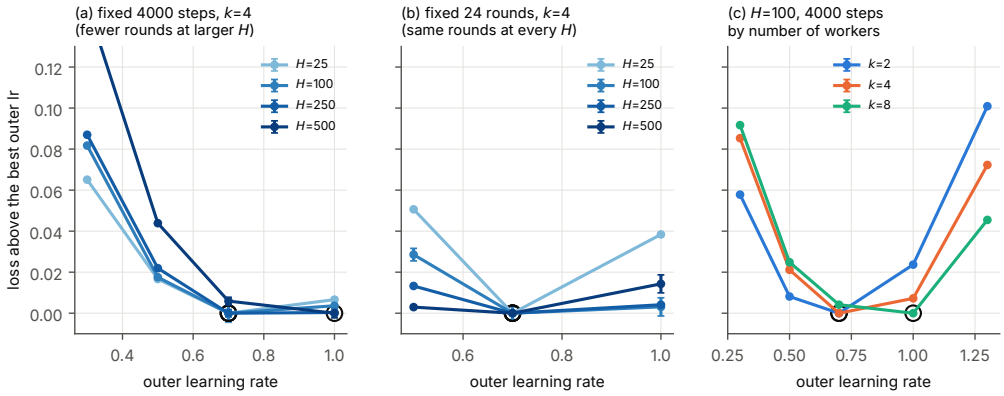


Figure 2. Loss above the best outer learning rate of each curve; circles mark the best value. (a) Fixed 4,000-step budget (seed means where available). (b) Every H given exactly 24 rounds; error bars over 3 seeds ($H = 100$) or 2 ($H = 500$). (c) $H = 100$, 4,000 steps, by number of workers (one seed).

3.3 C3: the best outer learning rate grows with the number of workers

At $H = 100$, the best value is 0.7 for 2 and 4 workers and 1.0 for 8 (Fig. 2c, Table 6). The margin at 8 workers (0.004) is within seed noise, but the whole curve shifts with k: a too-high value of 1.3 costs 0.101, 0.072 and 0.045 above the optimum for 2, 4 and 8 workers, while a too-low value of 0.3 costs 0.058, 0.085 and 0.092. The original optima for 2, 4 and 8 workers, read from their Figure 7, are roughly 0.6, 0.6 and 0.8.

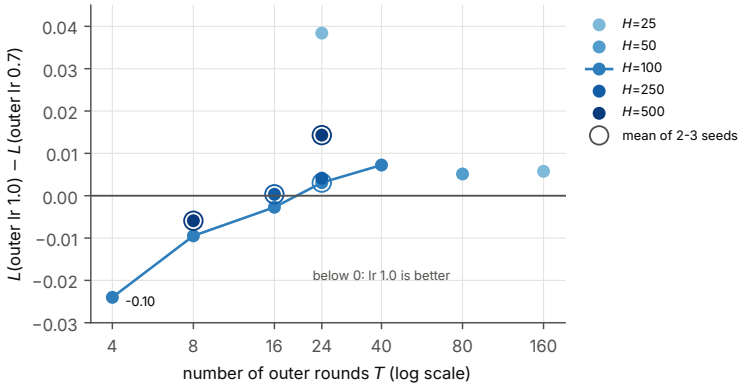


Figure 3. Loss with outer learning rate 1.0 minus loss with 0.7 ($k = 4$) against the number of outer rounds. Below zero, 1.0 is better. The line connects $H = 100$ runs with 4 to 40 rounds; rings mark seed-matched means over 2–3 seeds; the -0.10 point is drawn at the bottom edge.

3.4 C4: DiLoCo falls further behind data-parallel training with more workers

With outer learning rate 0.7 and a sync baseline of batch $32k$ for each k, DiLoCo at $H = 25$ ends 2.5%, 5.4% and 8.1% above the sync baseline for 2, 4 and 8 workers (Table 3), the same trend as the original 0.7%, 2.0% and 3.9%, with larger gaps at our shorter budget. Our 18,000-step run with 4 workers gives 3.7%, close to the original 2.0% at a similar tokens-per-parameter ratio. Measured as the share of the gap closed, the number of workers matters much less than H: 74%, 68% and 65% at $H = 25$; 28%, 33% and 33% at $H = 100$; 12%, 16% and 17% at $H = 500$ (Fig. 4).

Table 3. Final validation loss by number of workers, gap closed in parentheses (outer lr 0.7, 4,000 steps; $k = 4$ are 3-seed means). The solo baseline is 1.983.

k	Sync baseline	DiLoCo H=25	DiLoCo H=100	DiLoCo H=500
2	1.813	1.858 (74%)	1.936 (28%)	1.964 (12%)
4	1.697	1.789 (68%)	1.887 (33%)	1.936 (16%)
8	1.611	1.742 (65%)	1.861 (33%)	1.920 (17%)

3.5 C5: a single DiLoCo worker against AdamW

With $k = 1$, DiLoCo is AdamW with a Lookahead-style outer step [5] with Nesterov momentum, and the matching baseline is the solo baseline. At $H = 25$, close to the original $H = 30$, a single worker beats AdamW (1.970 against 1.983). At $H = 100$ and 500 it is 1.8% worse with either outer learning rate we tried (Table 4). With only 40 or 8 outer steps in this budget, the outer optimizer has little room to help, consistent with C2. The finding therefore holds here only at short sync intervals; we did not test whether it returns at longer budgets.

3.6 An additional observation: long sync intervals over longer training

The original study's sync intervals stop at $H = 300$, while Douillard et al.[1] used $H = 500$. In an 18,000-step run (about 20 tokens per parameter), $H = 25$ and 100 keep their benefit to the end, closing 70% and 55% of the gap, but $H = 500$, with only 36 rounds, fades from 47% after 4,000 steps to -4% at the end (Table 7, Fig. 5). Synchronous warm

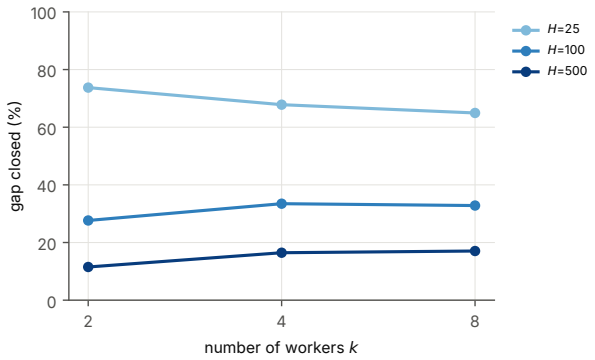


Figure 4. Share of the solo-to-sync gap closed by DiLoCo against the number of workers (outer lr 0.7, 4,000 steps).

Table 4. One DiLoCo worker against AdamW with the same batch and tokens (4,000 steps; DiLoCo one seed, AdamW 3 seeds).

H	Outer lr 0.4	Outer lr 0.7	Best vs. AdamW
25	1.970	1.980	-0.6%
100	2.024	2.019	+1.8%
500	2.097	2.018	+1.8%
AdamW (solo baseline)	1.983 ± 0.0009		–

starts of 2,000 or 6,000 steps restore it to 29% and 47%, at the cost of thousands of extra communication rounds. Douillard et al.[1] report that $H = 500$ works from scratch at larger scale with 128 rounds; given C2, the number of rounds is a plausible factor, but our runs cannot separate it from model size and data.

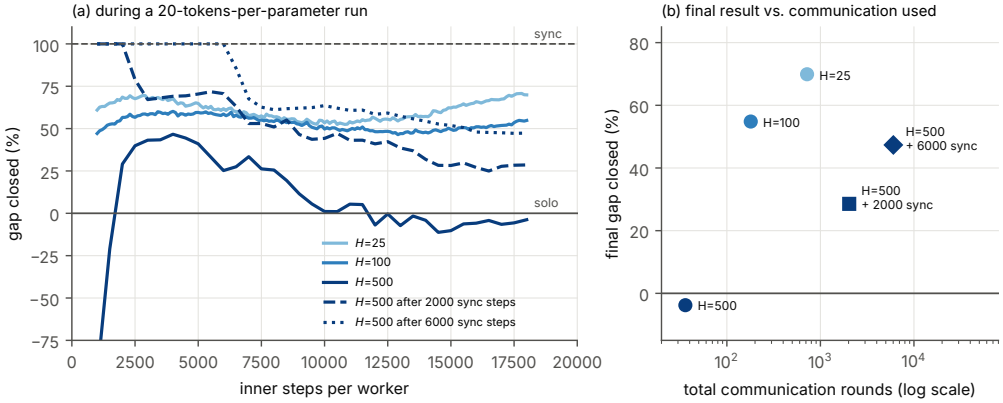


Figure 5. 18,000-step runs ($k = 4$, outer lr 0.7). (a) Gap closed during training. (b) Final gap closed against total communication rounds, counting warm-start steps as rounds.

4 Discussion

Four of the five findings replicate in a setting far smaller and cheaper than the original, which supports using small single-GPU proxies to study DiLoCo's hyperparameters. The places where our results depart from the original all trace back to the number of outer

rounds, which is small whenever a short budget meets a large H: it shifts the best outer learning rate (C2), it removes the advantage of a single worker at $H \geq 100$ (C5), and it is a plausible reason why $H = 500$ fails in our long runs. Practitioners using a small proxy should therefore report the number of rounds alongside H, and compare sync intervals at matched rounds as well as matched budgets.

What was easy. The method is simple to implement from its description, and the $k = 1$ equivalence with AdamW gives an exact correctness test. Simulating workers sequentially made eight workers fit on an 8 GB GPU.

What was difficult. The original study does not release code, and several details had to be chosen by us (data sharding, how the inner learning-rate schedule is indexed across rounds, the baseline batch size). Hyperparameter differences of 0.005–0.01 in loss, which decide C2 and C3, are close to seed noise at this scale and needed extra seeds. We also found and fixed an evaluation bug of our own during the study: the number of validation sequences depended on the training batch size, so large-batch baselines were scored on a superset of the validation set. All affected baselines were rerun; they moved by at most 0.005 (0.012 for a 600-step baseline), and a regression test now guards against it.

Communication with the original authors. We contacted the corresponding author of the original study on October 26, 2026, to share these results and the code; no response had been received at the time of submission.

Limitations. One architecture and one dataset that is simpler than web text; mostly 4 tokens per parameter; many configurations with one seed; no re-tuning of the inner learning rate; no wall-clock or bandwidth modelling.

5 Reproducing this work

All code, configurations and the logs of all 126 runs are in the repository. `python plot.py` regenerates every figure and table in this article from the logs in seconds, without a GPU. The full study takes about 71 GPU-hours on an RTX 3070 Ti. For review, `configs/review_quick.yaml` reruns the 12 runs behind the claims that rest on the smallest differences (C1, C2 at 4, 8 and 40 rounds, C5) in about 3 GPU-hours on an 8 GB GPU, and `compare_review.py` compares them with the released logs and checks each qualitative conclusion:

```
python data/prepare.py          # tokenize TinyStories (once)
python -m pytest tests/         # 10 correctness tests
python sweep.py configs/review_quick.yaml
python compare_review.py
```

bfloat16 kernels are not bit-for-bit deterministic across GPUs, so final losses may differ in the third decimal; differences above 0.015 are flagged.

Appendix: additional tables

Table 5. Every H trained for exactly 24 outer rounds ($k = 4$); baselines trained for the same number of steps. $\pm$: 3 seeds ($H = 100$) or 2 ($H = 500$). Gap closed in parentheses.

H	Steps	Sync	Solo	Outer lr 0.5	Outer lr 0.7	Outer lr 1.0
25	600	2.575	3.000	2.857 (34%)	2.806 (46%)	2.844 (37%)
100	2,400	1.830	2.175	2.093 $\pm$ 0.0030 (24%)	2.064 $\pm$ 0.0031 (32%)	2.067 $\pm$ 0.0044 (31%)
250	6,000	1.620	1.869	1.836 (13%)	1.823 (18%)	1.827 (17%)
500	12,000	1.520	1.723	1.734 (-5%)	1.731 $\pm$ 0.0001 (-4%)	1.745 $\pm$ 0.0044 (-11%)

Table 6. Outer learning rate by number of workers at $H = 100$ (4,000 steps, seed 0); best per k in bold.

k	lr 0.3	lr 0.5	lr 0.7	lr 1.0	lr 1.3
2	1.994	1.944	**1.936**	1.960	2.037
4	1.969	1.905	**1.884**	1.891	1.956
8	1.948	1.882	1.861	**1.857**	1.902

Table 7. Long-budget runs (18,000 steps, $k = 4$, outer lr 0.7, one seed).

Run	Comm. rounds	Val loss	Gap closed
H=25	720	1.535	70%
H=100	180	1.562	55%
H=500	36	1.668	-4%
H=500, after 2000 sync steps	2,032	1.610	29%
H=500, after 6000 sync steps	6,024	1.576	47%
Sync baseline	18,000	1.481	100%
Solo baseline	0	1.662	0%

References

1. A. Douillard, Q. Feng, A. A. Rusu, R. Chhaparia, Y. Donchev, A. Kuncoro, M. Ranzato, A. Szlam, and J. Shen. "DiLoCo: Distributed Low-Communication Training of Language Models." In: **arXiv preprint arXiv:2311.08105** (2023).
2. Z. Charles, G. Teston, L. Dery, K. Rush, N. Fallen, Z. Garrett, A. Szlam, and A. Douillard. "Communication-Efficient Language Model Training Scales Reliably and Robustly: Scaling Laws for DiLoCo." In: **Advances in Neural Information Processing Systems**. 2025. arXiv:2503.09799.
3. A. Karpathy. **nanoGPT**. 2022. URL: https://github.com/karpathy/nanoGPT.
4. R. Eldan and Y. Li. "TinyStories: How Small Can Language Models Be and Still Speak Coherent English?" In: **arXiv preprint arXiv:2305.07759** (2023).
5. M. R. Zhang, J. Lucas, G. Hinton, and J. Ba. "Lookahead Optimizer: k Steps Forward, 1 Step Back." In: **Advances in Neural Information Processing Systems**. Vol. 32. 2019.